

Maintenance & Future Evolution

Keeping FarmConnect Ghana Correct, Current, and Growing After Delivery

Covers all four ISO/IEC 14764 maintenance categories, the concrete bug-fixing and dependency-update process this repository already supports, and a staged roadmap — from a near-term hardening pass through to AI-driven and integration-led future phases.

Course	CSCD602 — Advanced Software Engineering, Capstone Project
Group	Angry Nerds
Project title	FarmConnect Ghana
Document version	1.0
Date	12 August 2026
Individual submission by	George Boadu Junior — 22427354

Table of Contents

1	Maintenance Categories	
2	Bug-Fixing Process	
3	Version Control & Release Strategy	
4	Dependency & Library Updates	
5	Security Update Process	
6	Scalability Plan	
7	Future Evolution Roadmap	
8	Technology Migration Candidates	
9	Future Integrations	
10	AI & Emerging-Technology Enhancements	

1. Maintenance Categories

Mapped against the concrete, already-documented gaps and deviations in the SRS (§9) and Testing Report (Defect Log, §11), rather than generic examples:

Category	Definition	Concrete example in this codebase
Corrective	Fixing defects discovered after delivery.	Defect D-03 (Testing Report) — the ~800KB single JS chunk — is exactly this kind of post-delivery fix: known, scoped, not yet applied. The stale <code>eslint-disable</code> directives (D-01/found-and-fixed) are a corrective fix already completed in this submission cycle.
Adaptive	Changing the system because its external environment changed.	Google has already retired dated Gemini model aliases (e.g. <code>gemini-2.5-flash</code>) for new API keys — <code>gemini.service.ts</code> deliberately uses the forward-compatible <code>gemini-flash-latest</code> alias specifically to absorb this kind of change without a code deploy. The next adaptive maintenance event is expected when Google eventually retires <i>that</i> alias in turn.
Perfective	Improving performance, maintainability, or usability without changing behaviour.	D-03's recommended fix (route-based code-splitting) is perfective — it changes nothing about what the app does, only how efficiently it loads. Widening automated frontend test coverage (Testing Report §11) is likewise perfective — it doesn't change behaviour, it changes confidence in behaviour.
Preventive	Changing the system before a problem manifests, to reduce future maintenance cost.	The Prisma migration history (7 migrations, one per phase) is itself a preventive practice — every schema change is reproducible and reversible instead of hand-edited on a live database. Adding automated dependency-vulnerability scanning (§5) before any CVE is actually exploited is the preventive posture this plan recommends next.

2. Bug-Fixing Process

- 1 Reproduce the bug locally against the same stack as production — Postgres 16 + Redis 7 via `docker-compose`, not an in-memory substitute, since several real defects in a system like this only appear against real Postgres/Redis behaviour (TTL expiry, transaction semantics, unique constraints).
- 2 Write a failing automated test first wherever the module already has test coverage (14 of the API's modules do) — this is what turns "we think we fixed it" into "we can prove we fixed it and it can't silently come back."
- 3 Fix the defect in the smallest diff that addresses the root cause — see the D-01 fix in the Testing Report (one line in `eslint.config.js`) as the model: find the actual root cause, not just the symptom, before touching code.
- 4 Re-run the full suite (`npm test` from the repo root) — a fix that breaks a different module is not a fix.
- 5 Record the defect (what, where found, severity, fix, verification) in a running defect log — the Testing Report's Defect Log table is the template to keep appending to, not a one-time report.
- 6 Deploy via the same path as every other change (§3) — no hand-editing files directly on a running server.

2.1 Severity & response targets

Severity	Definition	Target response
Critical	Data loss, security exposure, or the payment-reconciliation/dispute flow itself is broken.	Immediate hotfix, out of normal release cadence.
High	A core use case (UC-01–UC-07) is broken for a whole role.	Next scheduled release, expedited if no workaround exists.
Medium	A secondary feature (co-ops, ratings, SMS fallback) misbehaves without blocking the core flow.	Next scheduled release.
Low	Cosmetic, or a tooling/build-quality issue like D-01/D-03.	Batched with other low-severity fixes.

3. Version Control & Release Strategy

- **Repository:** Git, hosted on GitHub (`github.com/BoaduJnr/farmconnect-ghana`) — see `Links.txt/README` for the live link.
- **Branching:** `main` is always deployable; feature/fix work happens on short-lived branches merged via pull request, so every change has a reviewable diff and a point to revert to.
- **Migrations, not manual schema edits:** every schema change is a new Prisma migration file under `apps/api/prisma/migrations/` , applied with `npx prisma migrate deploy` — never a hand-run `ALTER TABLE` against production.
- **Environment parity:** the same `.env.example` keys are required in every environment (local, staging, production) — see the Deployment Guide for the authoritative list, so "works on my machine" configuration drift is structurally harder.
- **Rollback:** because releases are just new commits deployed the same way as every previous one, rolling back is redeploying the previous commit — no separate rollback tooling to maintain.

4. Dependency & Library Updates

What	Policy
Patch/minor updates	Run <code>npm outdated</code> monthly across all three workspaces; apply patch/minor bumps in a batch, then re-run the full test suite (\$Testing Report) before merging — low risk, high value, no reason to let these accumulate.
Major version bumps	One at a time, on its own branch, with the changelog read first — e.g. a future Express 5 or React 19 migration gets its own PR, not bundled with feature work.
Security advisories	<code>npm audit</code> (or GitHub's Dependabot alerts, once the repo is connected) reviewed on every dependency change; a flagged <i>direct</i> dependency is patched immediately regardless of the monthly cadence above.
Pinned/exact versions	<code>package-lock.json</code> is committed and is the source of truth for exactly what's installed — CI and every developer machine resolve to the same dependency tree.

5. Security Update Process

- **Secrets rotation** — `JWT_SECRET` / `JWT_REFRESH_SECRET` (≥ 32 chars, enforced by the zod-validated `env` loader) should be rotated on any suspected leak; rotating invalidates every existing refresh token at once (a deliberate, acceptable blast radius versus leaving a leaked secret live).
- **Dependency CVEs** — see §4; direct-dependency advisories are treated as security work, not routine maintenance.
- **Access-token exposure** — access/refresh tokens are currently stored in `localStorage` (SRS §9, Design Documentation "cross-cutting concerns"), a known XSS-exposure trade-off versus `httpOnly` cookies. **Recommended next security-hardening step:** migrate refresh-token storage to an `httpOnly`, `SameSite=Strict` cookie, keeping only the short-lived access token in memory.
- **Rate limiting** — OTP requests are already capped (20/hour/phone via Redis). **Recommended addition:** a coarser IP-based rate limit in front of the whole API once real traffic patterns are known, to blunt credential-stuffing-style abuse against the OTP endpoint itself.
- **Suspended-user tokens** — a suspended account's already-issued access token remains valid until its natural 12-hour expiry (a documented trade-off, SRS §9.3). If this is ever unacceptable for a real deployment, the fix is a short-TTL Redis-backed revocation check added to `requireAuth` — sized deliberately small so it can be added without redesigning auth.

6. Scalability Plan

Bottleneck (if traffic grows)	Planned response
API compute	The API is already stateless (session state lives in Redis/JWT, not in-process) — horizontal scaling is "run more instances behind a load balancer," not a redesign.
Database	Move from a single Postgres instance to a managed service with read replicas once read traffic (marketplace browsing) dominates; the Prisma layer already isolates the app from this change.
Listing search	The current in-application Haversine radius search (Design Documentation §6, "No PostGIS") is the first thing to replace under real load — either enable PostGIS on the existing Postgres or move geo-filtering to a dedicated search index (e.g. OpenSearch/Meilisearch) once listing volume makes an in-memory filter slow.
Real-time updates	REST polling (SRS §9.7) is the deliberate v1 choice; a WebSocket or push-notification upgrade is the natural next step once order/price update latency actually matters to users at scale.
Uploaded photos	Currently local disk (<code>apps/api/uploads/</code>) — the <code>.gitignore</code> already flags this as a "for this phase" choice; migrating to S3-compatible object storage removes the constraint that the API can only run on a single machine with persistent local disk.
Price-tick /notification fan-out	Both currently run as an in-process <code>setInterval</code> /inline call inside the single API instance; once there are multiple API instances, these move to a proper job queue (e.g. BullMQ over the existing Redis) so the tick/fan-out runs exactly once, not once per instance.

7. Future Evolution Roadmap

Phases 0–7 (README) delivered the complete v1 system this submission documents. The roadmap below is genuinely *next* — each phase sized to be independently shippable, in priority order.

Phase 8

Near-term

Hardening & observability

Close the two open Testing Report items (D-03 code-splitting; broader frontend component test coverage), add structured error tracking (e.g. Sentry) and a real uptime/health dashboard, and run a first load test to get an actual number against NFR-P2 instead of "not verified."

Phase 9

Real-time & push

Replace REST polling with WebSocket or Web Push for order-status changes and price alerts — the single highest-leverage UX upgrade once the SMS/offline-first constraints (still fully supported) aren't the only channel that matters.

Phase 10

Logistics & delivery tracking

The SRS explicitly excluded delivery dispatch management from v1 scope (§1.2). With the order/payment core proven, this is the natural next domain: connecting a listing's delivery fee to an actual rider/logistics partner instead of a flat fee with no tracking.

Phase 11

Regulated payments (optional)

If a real pilot outgrows manual Mobile Money reconciliation (SRS §9.1), integrate a licensed MoMo aggregator (e.g. a Paystack/Hubtel-style PSP) behind a feature flag — the `Order` model's seller/buyer momo fields and status machine were designed so this slots in without a schema rewrite, only a new payment-confirmation source.

Phase 12

Extension Officer role

Activate the already-reserved `EXTENSION` role (SRS §9.3) with real read-mostly aggregate views — regional listing/price trends, advisory-usage patterns — for agricultural extension staff and NGO partners named in the SRS's stakeholder list but never built for in v1.

8. Technology Migration Candidates

Candidate migration	Trigger / rationale
Local disk → S3-compatible object storage for uploads	Required as soon as the API needs to run as more than one instance (\$6).
In-process price tick/notifications → job queue (BullMQ)	Same trigger — required for correctness under horizontal scaling, not just nice-to-have.
REST polling → WebSocket/Web Push	User-experience driven (Phase 9 above), not forced by any current limitation.
Own simulated PriceFeed → a real Ghana Commodity Exchange (or equivalent) integration	Triggered the moment such a public, developer-accessible API actually exists (SRS §9.2) — the service boundary was deliberately kept thin for exactly this swap.
Express 4 → Express 5 / alternative framework	Only if/when Express 4 reaches end-of-support — no functional reason to migrate otherwise; treated as a major dependency bump (\$4), not an architecture change.

9. Future Integrations

- **A real Ghana Commodity Exchange (or equivalent) price API** — the top candidate, discussed in §8.
- **A licensed Mobile Money aggregator** for optional in-app payment (§7 Phase 11), kept opt-in so the zero-custody manual-reconciliation model remains available for users/regions that prefer it.
- **Logistics/delivery partners** (§7 Phase 10) — the `deliveryFee` field already exists on every order; integrating a real courier API is additive, not a schema change.
- **Agricultural input suppliers** — named in the original SRS's product context (§1.4) as a future adjacency (fertiliser/seed marketplace) but explicitly out of v1 scope; would reuse the existing Listing/Order machinery with a new participant type rather than a parallel system.
- **NGO/extension-officer reporting exports** — CSV/API exports of regional trends once the Extension role (Phase 12) is active.

10. AI & Emerging-Technology Enhancements

FarmConnect already ships one production AI feature (the Gemini-powered advisory chatbot with vision-based pest ID). The following are concrete, scoped extensions of that same capability — not speculative additions:

- **Proactive advisory push** — instead of only answering when asked, use the existing weather-context pipeline (already fetched per-conversation) to proactively notify farmers ahead of adverse weather for crops they've listed, via the existing notification channel.
- **Price-trend summarisation** — a short natural-language summary of what a farmer's crop's price has done over the last week/month, generated from the same `PriceSnapshot` history already being recorded, surfaced on the Prices tab.
- **Listing-quality assistant** — use Gemini's existing vision capability (already integrated for pest ID) to flag a listing photo that's blurry, poorly lit, or doesn't clearly show the produce, at upload time, before it goes live to buyers.
- **Fraud/anomaly signals for admins** — surface unusual patterns (e.g. a new account created and immediately disputing multiple payments) as a ranked queue in the Admin Portal, rather than the current chronological dispute list — a lightweight, explainable heuristic layer before reaching for a full ML model.
- **Voice input for search** — the one feature explicitly descope from v1.0's SRS (\$9.5); speech-to-text for the marketplace search bar would directly serve low-literacy users, the same population the SMS channel already targets.

GUIDING PRINCIPLE FOR ALL OF THE ABOVE

Every enhancement in this section extends a capability the system already has in production (Gemini, weather context, price history, the notification pipeline) rather than introducing a brand-new AI dependency — consistent with how the advisory chatbot itself was scoped in the first place (SRS \$9.4): pick the integration that's actually achievable and maintainable, not the most impressive-sounding one.